

Computational perspectives on the Classical Decision Problem

Lecture 1: Introduction

Ian Pratt-Hartmann
Manchester University

Bartosz Bednarczyk
Wrocław University

ESSLLI 2026

- Some formulas of first-order logic are **valid** (true in every structure), for example:

$$[\forall x(p(x) \rightarrow q(x)) \wedge \exists x(p(x) \wedge s(x))] \rightarrow \exists x(q(x) \wedge s(x)).$$

- Some formulas of first-order logic are **valid** (true in every structure), for example:

$$[\forall x(p(x) \rightarrow q(x)) \wedge \exists x(p(x) \wedge s(x))] \rightarrow \exists x(q(x) \wedge s(x)).$$

- Others are not. For example,

$$[\forall x(q(x) \rightarrow p(x)) \wedge \exists x(p(x) \wedge s(x))] \rightarrow \exists x(q(x) \wedge s(x))$$

is false in the some structures.

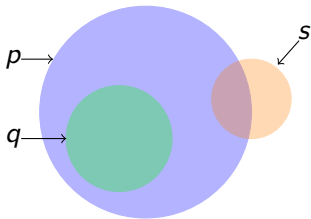
- Some formulas of first-order logic are **valid** (true in every structure), for example:

$$[\forall x(p(x) \rightarrow q(x)) \wedge \exists x(p(x) \wedge s(x))] \rightarrow \exists x(q(x) \wedge s(x)).$$

- Others are not. For example,

$$[\forall x(q(x) \rightarrow p(x)) \wedge \exists x(p(x) \wedge s(x))] \rightarrow \exists x(q(x) \wedge s(x))$$

is false in the some structures. For example:



- Suppose now we are given an arbitrary formula of first-order logic, e.g.

$$\forall x \forall y (x \neq y \rightarrow (r(x, y) \vee r(y, x))) \rightarrow \\ \exists x \forall y (x \neq y \rightarrow [r(x, y) \vee \exists z (r(x, z) \wedge r(z, y))]).$$

- Suppose now we are given an arbitrary formula of first-order logic, e.g.

$$\forall x \forall y (x \neq y \rightarrow (r(x, y) \vee r(y, x))) \rightarrow \\ \exists x \forall y (x \neq y \rightarrow [r(x, y) \vee \exists z (r(x, z) \wedge r(z, y))]).$$

- We ask: is this formula valid?

- Suppose now we are given an arbitrary formula of first-order logic, e.g.

$$\forall x \forall y (x \neq y \rightarrow (r(x, y) \vee r(y, x))) \rightarrow \\ \exists x \forall y (x \neq y \rightarrow [r(x, y) \vee \exists z (r(x, z) \wedge r(z, y))]).$$

- We ask: is this formula valid?
- More specifically, we are interested in how to solve this problem **algorithmically**.

- The quest for an algorithmic solution to determining validity in first-order logic is called the *Entscheidungsproblem* (decision problem):

Does there exist an algorithm to determine whether a given formula of first-order logic is valid?

- The quest for an algorithmic solution to determining validity in first-order logic is called the *Entscheidungsproblem* (decision problem):

Does there exist an algorithm to determine whether a given formula of first-order logic is valid?

- The question was first clearly posed by D. Hilbert and W. Ackermann: *Grundzüge der theoretischen Logik*, 1928.



- The quest for an algorithmic solution to determining validity in first-order logic is called the *Entscheidungsproblem* (decision problem):

Does there exist an algorithm to determine whether a given formula of first-order logic is valid?

- The question was first clearly posed by D. Hilbert and W. Ackermann: *Grundzüge der theoretischen Logik*, 1928.



- It was answered negatively by A. Church and A. Turing in 1936/7.

- The problem has a finite analogue: does there exist an algorithm to determine whether a given formula of first-order logic is **finitely** valid (true in every **finite** structure)?

- The problem has a finite analogue: does there exist an algorithm to determine whether a given formula of first-order logic is **finitely** valid (true in every **finite** structure)?
- This question is independent of the original *Entscheidungsproblem*, as it is not possible to write a formula φ_{fin} true in all and only those structures with finite domains.

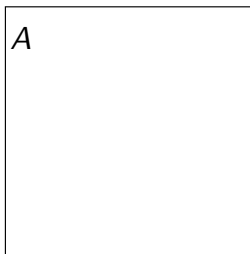
- The problem has a finite analogue: does there exist an algorithm to determine whether a given formula of first-order logic is **finitely** valid (true in every **finite** structure)?
- This question is independent of the original *Entscheidungsproblem*, as it is not possible to write a formula φ_{fin} true in all and only those structures with finite domains.
- It too was answered negatively by B. Trakhtenbrot in 1950.



- The following example provides a nice illustration of these problems.
 - Imagine a football league in which every team plays every other, under rules ensuring that each match produces a clear winner.
 - Define a *pretty good team* to be a team x such that, for any other team y , either x beats y , or x beats some team which beats y .
 - We claim that there must exist at least one pretty good team.

By induction on the number of teams in the league.

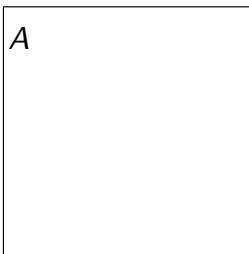
Let A be the set of teams in the league.



By induction on the number of teams in the league.

Let A be the set of teams in the league.

If $|A| = 1$, the statement is trivial.

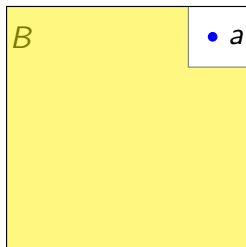


By induction on the number of teams in the league.

Let A be the set of teams in the league.

If $|A| = 1$, the statement is trivial.

Otherwise, pick $a \in A$, and let $B = A \setminus \{a\}$.



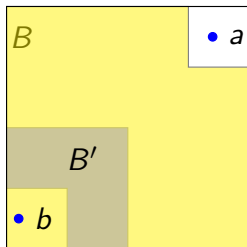
By induction on the number of teams in the league.

Let A be the set of teams in the league.

If $|A| = 1$, the statement is trivial.

Otherwise, pick $a \in A$, and let $B = A \setminus \{a\}$.

By IH, let b be a pretty good team with respect to B , and let B' be the set of teams in B that b beats.



By induction on the number of teams in the league.

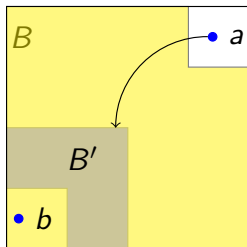
Let A be the set of teams in the league.

If $|A| = 1$, the statement is trivial.

Otherwise, pick $a \in A$, and let $B = A \setminus \{a\}$.

By IH, let b be a pretty good team with respect to B , and let B' be the set of teams in B that b beats.

If a beats every team in $B' \cup \{b\}$, then a is a pretty good team for A .



By induction on the number of teams in the league.

Let A be the set of teams in the league.

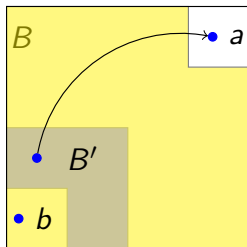
If $|A| = 1$, the statement is trivial.

Otherwise, pick $a \in A$, and let $B = A \setminus \{a\}$.

By IH, let b be a pretty good team with respect to B , and let B' be the set of teams in B that b beats.

If a beats every team in $B' \cup \{b\}$, then a is a pretty good team for A .

If some team in $B' \cup \{b\}$ beats a , then b is a pretty good team for A .



By induction on the number of teams in the league.

Let A be the set of teams in the league.

If $|A| = 1$, the statement is trivial.

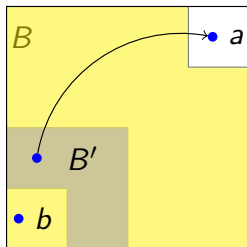
Otherwise, pick $a \in A$, and let $B = A \setminus \{a\}$.

By IH, let b be a pretty good team with respect to B , and let B' be the set of teams in B that b beats.

If a beats every team in $B' \cup \{b\}$, then a is a pretty good team for A .

If some team in $B' \cup \{b\}$ beats a , then b is a pretty good team for A .

This completes the induction.



- What this argument in fact shows is that the first-order formula

$$\forall x \forall y (x \neq y \rightarrow (r(x, y) \vee r(y, x))) \rightarrow \\ \exists x \forall y (x \neq y \rightarrow [r(x, y) \vee \exists z (r(x, z) \wedge r(z, y))])$$

is true in **every finite structure**.

- What this argument in fact shows is that the first-order formula

$$\forall x \forall y (x \neq y \rightarrow (r(x, y) \vee r(y, x))) \rightarrow \\ \exists x \forall y (x \neq y \rightarrow [r(x, y) \vee \exists z (r(x, z) \wedge r(z, y))])$$

is true in **every finite structure**.

- But this formula is not true in **every structure**: consider the natural numbers $\mathbb{N}$ for the domain of quantification, and let $r(x, y)$ be “ x is greater than y ”.

- What this argument in fact shows is that the first-order formula

$$\forall x \forall y (x \neq y \rightarrow (r(x, y) \vee r(y, x))) \rightarrow \\ \exists x \forall y (x \neq y \rightarrow [r(x, y) \vee \exists z (r(x, z) \wedge r(z, y))])$$

is true in **every finite structure**.

- But this formula is not true in **every structure**: consider the natural numbers $\mathbb{N}$ for the domain of quantification, and let $r(x, y)$ be “ x is greater than y ”.
- Thus, the formula is **finitely valid** but not **valid**.

- Usually, the *Entscheidungsproblem* is posed in a dual form.
- A sentence φ of first-order logic is **satisfiable** if, for some structure $\mathfrak{A}$, we have $\mathfrak{A} \models \varphi$.
- A sentence φ is **finitely satisfiable** if, for some **finite** structure $\mathfrak{A}$, we have $\mathfrak{A} \models \varphi$.

- Usually, the *Entscheidungsproblem* is posed in a dual form.
- A sentence φ of first-order logic is **satisfiable** if, for some structure $\mathfrak{A}$, we have $\mathfrak{A} \models \varphi$.
- A sentence φ is **finitely satisfiable** if, for some **finite** structure $\mathfrak{A}$, we have $\mathfrak{A} \models \varphi$.
- Our questions:
 - Does there exist an algorithm to determine whether a given sentence of first-order logic is **satisfiable**?
 - Does there exist an algorithm to determine whether it is **finitely satisfiable**?

- Usually, the *Entscheidungsproblem* is posed in a dual form.
- A sentence φ of first-order logic is **satisfiable** if, for some structure $\mathfrak{A}$, we have $\mathfrak{A} \models \varphi$.
- A sentence φ is **finitely satisfiable** if, for some **finite** structure $\mathfrak{A}$, we have $\mathfrak{A} \models \varphi$.
- Our questions:
 - Does there exist an algorithm to determine whether a given sentence of first-order logic is **satisfiable**?
 - Does there exist an algorithm to determine whether it is **finitely satisfiable**?
- This change is purely cosmetic: a sentence φ is (finitely) valid if and only if $\neg\varphi$ is not (finitely) valid.

- In fact, we can be slightly more general.
- A **formula** $\varphi(\bar{x})$ of first-order logic is **satisfiable** if, for some structure $\mathfrak{A}$ and tuple of objects $\bar{a}$ from A , we have $\mathfrak{A} \models \varphi[\bar{a}]$.
- A **formula** $\varphi(\bar{x})$ is **finitely satisfiable** if, for some **finite** structure $\mathfrak{A}$ and tuple $\bar{a}$, we have $\mathfrak{A} \models \varphi[\bar{a}]$.
- Our questions:
 - Does there exist an **algorithm** to determine whether a given **formula** of first-order logic is satisfiable?
 - Does there exist an algorithm to determine whether it is finitely satisfiable?
- This change is again cosmetic: a formula $\varphi(\bar{x})$ is (finitely) satisfiable if and only if the sentence $\exists \bar{x} \varphi$ is.

- When Hilbert and Ackermann formulated the *Entscheidungsproblem*, an algorithmic solution was already known for certain restricted classes of first-order formulas, or **fragments** of first-order logic:
 - The monadic fragment (L. Löwenheim, 1915)
 - Formulas $\exists \bar{x} \forall \bar{y} \psi$ (P. Bernays and M. Schönfinkel, 1928)
 - Formulas $\exists \bar{x} \forall y \exists \bar{z} \psi$ (W. Ackermann, 1928)
 - Equality-free formulas $\exists \bar{x} \forall y_1 \forall y_2 \exists \bar{z} \psi$ (K. Gödel, K. Schütte, L. Kalmár 1933).

- When Hilbert and Ackermann formulated the *Entscheidungsproblem*, an algorithmic solution was already known for certain restricted classes of first-order formulas, or **fragments** of first-order logic:
 - The monadic fragment (L. Löwenheim, 1915)
 - Formulas $\exists \bar{x} \forall \bar{y} \psi$ (P. Bernays and M. Schönfinkel, 1928)
 - Formulas $\exists \bar{x} \forall y \exists \bar{z} \psi$ (W. Ackermann, 1928)
 - Equality-free formulas $\exists \bar{x} \forall y_1 \forall y_2 \exists \bar{z} \psi$ (K. Gödel, K. Schütte, L. Kalmár 1933).
- All these fragments include the oldest fragment of all: the Aristotelian syllogistic.

- When Hilbert and Ackermann formulated the *Entscheidungsproblem*, an algorithmic solution was already known for certain restricted classes of first-order formulas, or **fragments** of first-order logic:
 - The monadic fragment (L. Löwenheim, 1915)
 - Formulas $\exists \bar{x} \forall \bar{y} \psi$ (P. Bernays and M. Schönfinkel, 1928)
 - Formulas $\exists \bar{x} \forall y \exists \bar{z} \psi$ (W. Ackermann, 1928)
 - Equality-free formulas $\exists \bar{x} \forall y_1 \forall y_2 \exists \bar{z} \psi$ (K. Gödel, K. Schütte, L. Kalmár 1933).
- All these fragments include the oldest fragment of all: the Aristotelian syllogistic.
- Thus, the *Entscheidungsproblem* becomes a **classification problem**: which fragments of first-order logic are decidable for validity?

- If $\mathcal{L}$ is any fragment of first-order logic, the **satisfiability problem** for $\mathcal{L}$ is the following problem:

Sat($\mathcal{L}$)

Given: a finite set Φ of $\mathcal{L}$ -formulas

Return: Yes if Φ is satisfiable

No otherwise.

- The **finite satisfiability problem** for $\mathcal{L}$ is the following problem:

FinSat($\mathcal{L}$)

Given: a finite set Φ of $\mathcal{L}$ -formulas

Return: Yes if Φ is finitely satisfiable

No otherwise.

- Most fragments of interest are closed under conjunction. In that case, we may take the input to Sat($\mathcal{L}$) or FinSat($\mathcal{L}$) to be a single formula φ .
- We ask: for which fragments $\mathcal{L}$ of first-order logic are Sat($\mathcal{L}$) and/or FinSat($\mathcal{L}$) decidable.

- A fragment $\mathcal{L}$ of first-order logic is said to have the **finite model property** if all satisfiable finite sets of formulas are finitely satisfiable, or, equivalently, if $\text{Sat}(\mathcal{L}) = \text{FinSat}(\mathcal{L})$.
- In this course, we shall discuss the satisfiability (and finite satisfiability) problems for the following fragments:
 - two-variable fragment;
 - the guarded fragment;
 - the Gödel fragment;
 - fragments with counting quantifiers;
 - variable-ordering fragments (the ‘fluted’ fragment).
- We shall present algorithms to solve these problems, and show how to derive bounds on their computational complexity.
- Where the fragments in question have the finite model property, we shall establish this too.

- Plan of the course:

- Lecture 1: Introduction to the *Entscheidungsproblem*
- Lecture 2: The monadic fragment.
- Lecture 3: The two-variable fragment.
- Lecture 4: The guarded fragment
- Lecture 5: The Gödel fragment
- Lecture 6: The one-variable fragment with counting
- Lecture 7: The two-variable fragment with counting
- Lecture 8: The guarded fragment and the FMP
- Lecture 9: The fluted fragment
- Lecture 10: A glimpse beyond ...

- What you will need for this course:
 - Basic first-order logic: formulas, structures, satisfaction.
 - Basic computability theory: Turing machines, the notions of recursive, recursively enumerable and co-recursively enumerable sets.
 - Basic complexity theory: deterministic and non-deterministic Turing machines, alternating Turing machines, standard time- and space-complexity bounds, reductions, hardness and completeness.
- A more detailed list follows . . .

- We assume familiarity with the basis of first-order logic.

- We assume familiarity with the basis of first-order logic.
- Structures, domains and elements:
 - structures: $\mathfrak{A}, \mathfrak{B} \dots$
 - their domains: $A, B, \dots$
 - elements of domains: $a, b, \dots$

Domains will always be assumed to be non-empty.

- We assume familiarity with the basis of first-order logic.
- Structures, domains and elements:
 - structures: $\mathfrak{A}, \mathfrak{B} \dots$
 - their domains: $A, B, \dots$
 - elements of domains: $a, b, \dots$

Domains will always be assumed to be non-empty.

- Signatures:
 - predicates (of non-zero arity): $p, q, r, s, \dots$
 - proposition letters (0-ary predicates): $P, Q, \dots$
 - individual constants: $c, d, \dots$
 - function-symbols: $f, g, \dots$

The (logical) equality predicate $=$ is available by default.

- We assume familiarity with the basis of first-order logic.
- Structures, domains and elements:
 - structures: $\mathfrak{A}, \mathfrak{B} \dots$
 - their domains: $A, B, \dots$
 - elements of domains: $a, b, \dots$

Domains will always be assumed to be non-empty.

- Signatures:
 - predicates (of non-zero arity): $p, q, r, s, \dots$
 - proposition letters (0-ary predicates): $P, Q, \dots$
 - individual constants: $c, d, \dots$
 - function-symbols: $f, g, \dots$

The (logical) equality predicate $=$ is available by default.

- Suppose $\varphi(\bar{x})$ is a formula, $\mathfrak{A}$ a structure interpreting its signature, and $\bar{a}$ a tuple of elements of A . We write $\mathfrak{A} \models \varphi[\bar{a}]$ to mean that φ is satisfied in $\mathfrak{A}$ under the assignment $\bar{x} \leftarrow \bar{a}$.

- We assume familiarity with the basis of first-order logic.
- Structures, domains and elements:
 - structures: $\mathfrak{A}, \mathfrak{B} \dots$
 - their domains: $A, B, \dots$
 - elements of domains: $a, b, \dots$

Domains will always be assumed to be non-empty.

- Signatures:
 - predicates (of non-zero arity): $p, q, r, s, \dots$
 - proposition letters (0-ary predicates): $P, Q, \dots$
 - individual constants: $c, d, \dots$
 - function-symbols: $f, g, \dots$

The (logical) equality predicate $=$ is available by default.

- Suppose $\varphi(\bar{x})$ is a formula, $\mathfrak{A}$ a structure interpreting its signature, and $\bar{a}$ a tuple of elements of A . We write $\mathfrak{A} \models \varphi[\bar{a}]$ to mean that φ is satisfied in $\mathfrak{A}$ under the assignment $\bar{x} \leftarrow \bar{a}$.
- A **sentence** is a formula with no free variables.

- We assume familiarity with the basis of first-order logic.
- Structures, domains and elements:
 - structures: $\mathfrak{A}, \mathfrak{B} \dots$
 - their domains: $A, B, \dots$
 - elements of domains: $a, b, \dots$

Domains will always be assumed to be non-empty.

- Signatures:
 - predicates (of non-zero arity): $p, q, r, s, \dots$
 - proposition letters (0-ary predicates): $P, Q, \dots$
 - individual constants: $c, d, \dots$
 - function-symbols: $f, g, \dots$

The (logical) equality predicate $=$ is available by default.

- Suppose $\varphi(\bar{x})$ is a formula, $\mathfrak{A}$ a structure interpreting its signature, and $\bar{a}$ a tuple of elements of A . We write $\mathfrak{A} \models \varphi[\bar{a}]$ to mean that φ is satisfied in $\mathfrak{A}$ under the assignment $\bar{x} \leftarrow \bar{a}$.
- A **sentence** is a formula with no free variables.
- We denote the set of first-order formulas by $\mathcal{FO}$.

- We assume familiarity with the basics of computability.

- We assume familiarity with the basics of computability.
- We can think of any decision problem as a set of strings.

- We assume familiarity with the basics of computability.
- We can think of any decision problem as a set of strings.
- A set (of strings) L is said to be **computable** or **recursive** if there is some Turing machine M that is guaranteed to halt, and that accepts (terminates successfully on) input x if and only if $x \in L$.

- We assume familiarity with the basics of computability.
- We can think of any decision problem as a set of strings.
- A set (of strings) L is said to be **computable** or **recursive** if there is some Turing machine M that is guaranteed to halt, and that accepts (terminates successfully on) input x if and only if $x \in L$.
- A set L is said to be **recursively enumerable** (r.e.) if there is some Turing machine M that accepts input x if and only if $x \in L$.
- A set L is said to be **co-recursively enumerable** (co-r.e.) if there is some Turing machine M that accepts input x if and only if $x \notin L$.
- A set is **recursive** iff it is r.e. and co-r.e.

- We assume familiarity with the basics of computability.
- We can think of any decision problem as a set of strings.
- A set (of strings) L is said to be **computable** or **recursive** if there is some Turing machine M that is guaranteed to halt, and that accepts (terminates successfully on) input x if and only if $x \in L$.
- A set L is said to be **recursively enumerable** (r.e.) if there is some Turing machine M that accepts input x if and only if $x \in L$.
- A set L is said to be **co-recursively enumerable** (co-r.e.) if there is some Turing machine M that accepts input x if and only if $x \notin L$.
- A set is **recursive** iff it is r.e. and co-r.e.
- We follow standard practice and equate algorithmic decidability of a problem with the property of being recursive (**Church-Turing hypothesis**).

- In 1930, K. Gödel showed that the set of valid sentences of first-order logic can be **axiomatically characterized** in terms of a finite list of axiom schemas and the inference rules of generalization and *modus ponens*.

- In 1930, K. Gödel showed that the set of valid sentences of first-order logic can be **axiomatically characterized** in terms of a finite list of axiom schemas and the inference rules of generalization and *modus ponens*.
- Now consider the problems $\text{Sat}(\mathcal{FO})$ and $\text{FinSat}(\mathcal{FO})$.

- In 1930, K. Gödel showed that the set of valid sentences of first-order logic can be **axiomatically characterized** in terms of a finite list of axiom schemas and the inference rules of generalization and *modus ponens*.
- Now consider the problems $\text{Sat}(\mathcal{FO})$ and $\text{FinSat}(\mathcal{FO})$.
- If the answer to some instance φ of $\text{FinSat}(\mathcal{FO})$ is Yes, then we can eventually discover this (try each finite structure).

- In 1930, K. Gödel showed that the set of valid sentences of first-order logic can be **axiomatically characterized** in terms of a finite list of axiom schemas and the inference rules of generalization and *modus ponens*.
- Now consider the problems $\text{Sat}(\mathcal{FO})$ and $\text{FinSat}(\mathcal{FO})$.
- If the answer to some instance φ of $\text{FinSat}(\mathcal{FO})$ is Yes, then we can eventually discover this (try each finite structure).
- If the answer to some instance φ of $\text{Sat}(\mathcal{FO})$ is No, then we can eventually discover this (enumerate the proofs and see if you get $\neg\varphi$).

- In 1930, K. Gödel showed that the set of valid sentences of first-order logic can be **axiomatically characterized** in terms of a finite list of axiom schemas and the inference rules of generalization and *modus ponens*.
- Now consider the problems $\text{Sat}(\mathcal{FO})$ and $\text{FinSat}(\mathcal{FO})$.
- If the answer to some instance φ of $\text{FinSat}(\mathcal{FO})$ is Yes, then we can eventually discover this (try each finite structure).
- If the answer to some instance φ of $\text{Sat}(\mathcal{FO})$ is No, then we can eventually discover this (enumerate the proofs and see if you get $\neg\varphi$).
- Hence $\text{FinSat}(\mathcal{FO})$ is in r.e. and $\text{Sat}(\mathcal{FO})$ is in co-r.e.

- In 1930, K. Gödel showed that the set of valid sentences of first-order logic can be **axiomatically characterized** in terms of a finite list of axiom schemas and the inference rules of generalization and *modus ponens*.
- Now consider the problems $\text{Sat}(\mathcal{FO})$ and $\text{FinSat}(\mathcal{FO})$.
- If the answer to some instance φ of $\text{FinSat}(\mathcal{FO})$ is Yes, then we can eventually discover this (try each finite structure).
- If the answer to some instance φ of $\text{Sat}(\mathcal{FO})$ is No, then we can eventually discover this (enumerate the proofs and see if you get $\neg\varphi$).
- Hence $\text{FinSat}(\mathcal{FO})$ is in r.e. and $\text{Sat}(\mathcal{FO})$ is in co-r.e.
- Furthermore, if a fragment $\mathcal{L}$ of $\mathcal{FO}$ has the FMP, then $\text{Sat}(\mathcal{L})$ coincides with $\text{FinSat}(\mathcal{L})$, and is decidable.

- A problem P is **r.e.-hard** if any problem in r.e. can be computably reduced to P . Similarly for **co.-r.e.-hard**.
- Turing showed that the set of **satisfiable** formulas of first-order logic is co-r.e.-hard.

- A problem P is **r.e.-hard** if any problem in r.e. can be computably reduced to P . Similarly for **co.-r.e.-hard**.
- Turing showed that the set of **satisfiable** formulas of first-order logic is co-r.e.-hard.
- Trakhtenbrot showed that the set of **finitely satisfiable** formulas of first-order logic is r.e.-hard.

- A problem P is **r.e.-hard** if any problem in r.e. can be computably reduced to P . Similarly for **co.-r.e.-hard**.
- Turing showed that the set of **satisfiable** formulas of first-order logic is co-r.e.-hard.
- Trakhtenbrot showed that the set of **finitely satisfiable** formulas of first-order logic is r.e.-hard.
- A problem P is **r.e.-complete** if it is r.e. and r.e.-hard. Similarly for **co.-r.e.-complete**.

- A problem P is **r.e.-hard** if any problem in r.e. can be computably reduced to P . Similarly for **co-r.e.-hard**.
- Turing showed that the set of **satisfiable** formulas of first-order logic is co-r.e.-hard.
- Trakhtenbrot showed that the set of **finitely satisfiable** formulas of first-order logic is r.e.-hard.
- A problem P is **r.e.-complete** if it is r.e. and r.e.-hard. Similarly for **co-r.e.-complete**.
- Thus, we have:

Theorem

The problem $\text{Sat}(\mathcal{FO})$ is co-r.e.-complete.

The problem $\text{FinSat}(\mathcal{FO})$ is r.e.-complete.

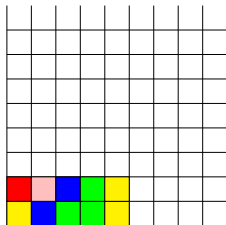
- A standard way to show these hardness results is in terms of domino problems (Hao Wang, 1960).

- A standard way to show these hardness results is in terms of domino problems (Hao Wang, 1960).
- A tiling system is a triple (C, H, V) , where C is a finite set, and H, V are binary relations over C .
 - We call the elements of C colours
 - we call H and V the horizontal and vertical constraints, respectively.

- A standard way to show these hardness results is in terms of **domino problems** (Hao Wang, 1960).
- A **tiling system** is a triple (C, H, V) , where C is a finite set, and H, V are binary relations over C .
 - We call the elements of C **colours**
 - we call H and V the **horizontal** and **vertical constraints**, respectively.
- A **(C, H, V) -tiling of $\mathbb{N}^2$** is a function $t : \mathbb{N}^2 \rightarrow C$ such that, for all $i, j \in \mathbb{N}$:
 - $\langle t(i, j), t(i + 1, j) \rangle \in H$; and
 - $\langle t(i, j), t(i, j + 1) \rangle \in V$.

- A standard way to show these hardness results is in terms of **domino problems** (Hao Wang, 1960).
- A **tiling system** is a triple (C, H, V) , where C is a finite set, and H, V are binary relations over C .
 - We call the elements of C **colours**
 - we call H and V the **horizontal** and **vertical constraints**, respectively.
- A **(C, H, V) -tiling of $\mathbb{N}^2$** is a function $t : \mathbb{N}^2 \rightarrow C$ such that, for all $i, j \in \mathbb{N}$:
 - $\langle t(i, j), t(i + 1, j) \rangle \in H$; and
 - $\langle t(i, j), t(i, j + 1) \rangle \in V$.

- Think of tilings in the obvious way:



- The **constrained infinite tiling problem** is as follows:

Given: A tiling system (C, H, V) and some $c_\alpha \in C$

Return: Yes if there is a (C, H, V) -tiling t of $\mathbb{N}^2$ s.t. $t(0, 0) = c_\alpha$
No otherwise.

- The **constrained finite tiling problem** is as follows:

Given: A tiling system (C, H, V) and $c_\alpha, c_\omega \in C$

Return: Yes if there is a (C, H, V) -tiling t of $[0, n]^2$ for some
 n , s.t. $t(0, 0) = c_\alpha$ and $t(n, n) = c_\omega$
No otherwise.

- If the answer to some instance of CFT is Yes, then we can eventually discover this (just try them one by one).

- If the answer to some instance of CFT is Yes, then we can eventually discover this (just try them one by one).
- If the answer to some instance of CIT is No, then we can eventually discover this (because we will get stuck at some point).

- If the answer to some instance of CFT is Yes, then we can eventually discover this (just try them one by one).
- If the answer to some instance of CIT is No, then we can eventually discover this (because we will get stuck at some point).
- Thus, CFT is in r.e., and CIT is in co-r.e.

- If the answer to some instance of CFT is Yes, then we can eventually discover this (just try them one by one).
- If the answer to some instance of CIT is No, then we can eventually discover this (because we will get stuck at some point).
- Thus, CFT is in r.e., and CIT is in co-r.e.
- It is possible to show:

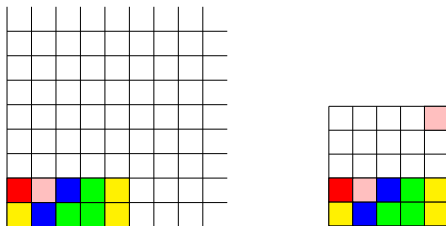
Theorem

The problem CIT is co-r.e.-complete.

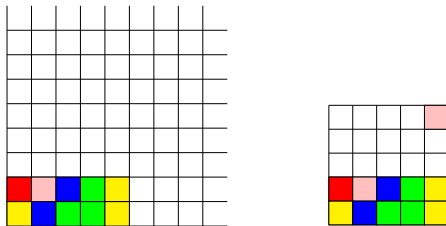
The problem CFT is r.e.-complete.

- Tiling problems are typically easier to work with than Turing machines.

- Tiling problems are typically easier to work with than Turing machines.
- In particular, it is easy to reduce $\text{FinSat}(\mathcal{FO})$ to the problem CFT, and to reduce $\text{Sat}(\mathcal{FO})$ to the problem CIT.



- Tiling problems are typically easier to work with than Turing machines.
- In particular, it is easy to reduce $\text{FinSat}(\mathcal{FO})$ to the problem CFT, and to reduce $\text{Sat}(\mathcal{FO})$ to the problem CIT.



- Such reductions are also frequently available for certain fragments $\mathcal{L}$ of $\mathcal{FO}$.

- We assume familiarity with the basics of computational complexity theory.

- We assume familiarity with the basics of computational complexity theory.
- We have the familiar hierarchy of complexity classes:

$$\begin{aligned} \text{NLOGSPACE} \subseteq \text{PTIME} \subseteq \text{NPTIME} \subseteq \\ \text{PSPACE} \subseteq \text{EXPTIME} \subseteq \text{NEXPTIME} \subseteq \\ 2\text{-EXPTIME} \subseteq 2\text{-NEXPTIME} \dots \end{aligned}$$

- We assume familiarity with the basics of computational complexity theory.
- We have the familiar hierarchy of complexity classes:

$$\begin{aligned} \text{NLOGSPACE} \subseteq \text{PTIME} \subseteq \text{NPTIME} \subseteq \\ \text{PSPACE} \subseteq \text{EXPTIME} \subseteq \text{NEXPTIME} \subseteq \\ 2\text{-EXPTIME} \subseteq 2\text{-NEXPTIME} \dots \end{aligned}$$

- For any complexity class $\mathcal{C}$, a problem P is $\mathcal{C}$ -hard if, any other problem Q in $\mathcal{C}$ can be reduced to it (many-one log-space reduction).
- A problem P is $\mathcal{C}$ -complete if it is in $\mathcal{C}$ and $\mathcal{C}$ -hard.

- We assume familiarity with the basics of computational complexity theory.
- We have the familiar hierarchy of complexity classes:

$$\begin{aligned} \text{NLOGSPACE} \subseteq \text{PTIME} \subseteq \text{NPTIME} \subseteq \\ \text{PSPACE} \subseteq \text{EXPTIME} \subseteq \text{NEXPTIME} \subseteq \\ 2\text{-EXPTIME} \subseteq 2\text{-NEXPTIME} \dots \end{aligned}$$

- For any complexity class $\mathcal{C}$, a problem P is $\mathcal{C}$ -hard if, any other problem Q in $\mathcal{C}$ can be reduced to it (many-one log-space reduction).
- A problem P is $\mathcal{C}$ -complete if it is in $\mathcal{C}$ and $\mathcal{C}$ -hard.
- When we speak of problems being r.e.-hard (-complete) or co-r.e.-hard (-complete), we generally have in mind **computable** reducibility.

- We formulated Hilbert and Ackermann's **Entscheidungsproblem** (decision problem) for first-order logic. We also defined its finite analogue.

- We formulated Hilbert and Ackermann's **Entscheidungsproblem** (decision problem) for first-order logic. We also defined its finite analogue.
- We explained that **Church** and **Turing** showed that there is no algorithm for determining satisfiability in $\mathcal{FO}$. **Trakhtenbrot** did the same for finite satisfiability.

- We formulated Hilbert and Ackermann's **Entscheidungsproblem** (decision problem) for first-order logic. We also defined its finite analogue.
- We explained that **Church** and **Turing** showed that there is no algorithm for determining satisfiability in $\mathcal{FO}$. **Trakhtenbrot** did the same for finite satisfiability.
- These problems are, however, decidable for **fragments** of first-order logic.

- We formulated Hilbert and Ackermann's **Entscheidungsproblem** (decision problem) for first-order logic. We also defined its finite analogue.
- We explained that **Church** and **Turing** showed that there is no algorithm for determining satisfiability in $\mathcal{FO}$. **Trakhtenbrot** did the same for finite satisfiability.
- These problems are, however, decidable for **fragments** of first-order logic.
- Therefore, the decision problem becomes a **classification problem**: for which fragments of $\mathcal{FO}$ is the (finite) satisfiability problem decidable, and what is its complexity?

- We formulated Hilbert and Ackermann's **Entscheidungsproblem** (decision problem) for first-order logic. We also defined its finite analogue.
- We explained that **Church** and **Turing** showed that there is no algorithm for determining satisfiability in $\mathcal{FO}$. **Trakhtenbrot** did the same for finite satisfiability.
- These problems are, however, decidable for **fragments** of first-order logic.
- Therefore, the decision problem becomes a **classification problem**: for which fragments of $\mathcal{FO}$ is the (finite) satisfiability problem decidable, and what is its complexity?
- We defined the **finite model property (FMP)** and showed that it implies decidability of satisfiability.

- We formulated Hilbert and Ackermann's **Entscheidungsproblem** (decision problem) for first-order logic. We also defined its finite analogue.
- We explained that **Church** and **Turing** showed that there is no algorithm for determining satisfiability in $\mathcal{FO}$. **Trakhtenbrot** did the same for finite satisfiability.
- These problems are, however, decidable for **fragments** of first-order logic.
- Therefore, the decision problem becomes a **classification problem**: for which fragments of $\mathcal{FO}$ is the (finite) satisfiability problem decidable, and what is its complexity?
- We defined the **finite model property (FMP)** and showed that it implies decidability of satisfiability.
- We briefly described constrained (infinite and finite) **tiling problems** and explained that these are often easier to think about than Turing machines.